

COMPTE RENDU D'EXAMEN PRATIQUE

Simulation Arduino sous Proteus ISIS

Détection analogique, feu tricolore, commande manuelle et
servomoteur

Nom et prénom : [À compléter]

.....

Année universitaire : [À compléter]

.....

Enseignant : M. Aymane BOUMEZZOUGH

.....

Date : [À compléter]

.....

Sommaire

1. Introduction
2. Exercice 1 — Détecteur de seuil analogique
3. Exercice 2 — Gestion d'un feu tricolore
4. Exercice 3 — Commande manuelle d'éclairage
5. Exercice 4 — Contrôle angulaire d'un servomoteur
6. Difficultés rencontrées
7. Conclusion générale

Introduction

Ce travail pratique porte sur la programmation d'une carte Arduino Uno et sur la vérification de son fonctionnement dans Proteus ISIS. Les quatre exercices abordent des situations simples d'instrumentation : lire une grandeur analogique, commander des sorties numériques avec une temporisation, lire un bouton-poussoir et positionner un servomoteur.

Les programmes sont écrits et compilés dans Arduino IDE. Le fichier HEX obtenu est ensuite chargé dans le composant Arduino Uno du schéma Proteus. Cette méthode permet d'observer le comportement du montage avant une éventuelle réalisation physique.

Les objectifs du TP sont les suivants :

- utiliser les entrées analogiques et numériques de l'Arduino ;
- piloter des LED en respectant un état et une durée imposés ;
- comprendre le rôle d'une résistance pull-down ;
- convertir une mesure analogique en consigne angulaire avec `map ( )` ;
- vérifier la cohérence entre le câblage, le programme et les résultats.

Note avant remise : les cadres marqués « À insérer » et « À compléter » doivent être remplacés par les captures et observations obtenues réellement dans Proteus. Ce document ne prétend pas qu'une simulation a été exécutée.

Exercice 1 — Détecteur de seuil analogique

Objectif : allumer une LED rouge uniquement lorsque la valeur mesurée sur l'entrée A0 est strictement supérieure à 512.

1. Composants et connexions

Le montage utilise un Arduino Uno R3, un potentiomètre POT-HG, une LED rouge et une résistance MINIRES de 220 Ω pour limiter le courant de la LED.

Élément	Connexion Proteus
Potentiomètre	Bornes extérieures vers 5 V et GND ; curseur vers A0
LED rouge	Broche 8 → résistance 220 Ω → anode ; cathode → GND

2. Schéma électrique

Capture du schéma Proteus à insérer ici

La capture doit montrer l'Arduino Uno, A0, la broche 8, le potentiomètre, la LED, la résistance, le 5 V et le GND.

3. Programme Arduino commenté

```
const int potentiometerPin = A0;
const int ledPin = 8;

void setup() {
  // La LED est commandée par la broche 8.
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // Lecture du potentiomètre entre 0 et 1023.
  int analogValue = analogRead(potentiometerPin);

  // La LED s'allume seulement au-dessus du seuil 512.
  if (analogValue > 512) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }
}
```

4. Fonctionnement du programme

La fonction `analogRead(A0)` produit une valeur entière comprise entre 0 et 1023. À chaque passage dans `loop()`, le programme compare cette mesure au seuil 512. Si la condition `analogValue > 512` est vraie, la sortie 8 passe à l'état HIGH et la LED s'allume. Sinon, la sortie reste à LOW et la LED s'éteint. La valeur 512 elle-même appartient donc au cas « LED éteinte ».

5. Résultats attendus

- mesure comprise entre 0 et 512 : LED éteinte ;
- mesure comprise entre 513 et 1023 : LED allumée ;
- le changement d'état se produit au franchissement du seuil.

6. Résultats expérimentaux et analyse

Capture de simulation à insérer ici

Montrer au moins un état significatif du potentiomètre et de la LED.

Observations réelles à compléter :

Valeur ou position testée : [...]

État observé de la LED : [...]

Comparaison avec le résultat attendu : [...]

7. Conclusion de l'exercice

Cet exercice met en œuvre une lecture analogique et une décision simple par seuil. Le programme respecte la condition stricte demandée : 512 n'allume pas la LED. La simulation finale devra confirmer le passage de l'état éteint à l'état allumé à partir de 513. Difficulté éventuellement rencontrée : **[à compléter après la simulation]**.

Exercice 2 — Gestion d'un feu tricolore

Objectif : reproduire en boucle un cycle de signalisation composé de 3 s de vert, 1 s de jaune et 3 s de rouge.

1. Composants et connexions

Le montage contient un Arduino Uno R3, une LED rouge, une LED jaune, une LED verte et trois résistances MINIRES de 220 Ω .

Feu	Sortie Arduino	Connexion
Rouge	13	D13 $\rightarrow$ 220 Ω $\rightarrow$ anode ; cathode $\rightarrow$ GND
Jaune	12	D12 $\rightarrow$ 220 Ω $\rightarrow$ anode ; cathode $\rightarrow$ GND
Vert	11	D11 $\rightarrow$ 220 Ω $\rightarrow$ anode ; cathode $\rightarrow$ GND

2. Schéma électrique

Capture du schéma Proteus à insérer ici

La capture doit rendre lisibles les broches 13, 12 et 11, les trois LED, leurs résistances et le GND commun.

3. Programme Arduino commenté

```
const int redLedPin = 13;
const int yellowLedPin = 12;
const int greenLedPin = 11;

void setup() {
  pinMode(redLedPin, OUTPUT);
  pinMode(yellowLedPin, OUTPUT);
  pinMode(greenLedPin, OUTPUT);
}

void loop() {
  // Feu vert pendant 3 secondes.
  digitalWrite(greenLedPin, HIGH);
  digitalWrite(yellowLedPin, LOW);
  digitalWrite(redLedPin, LOW);
  delay(3000);

  // Feu jaune pendant 1 seconde.
  digitalWrite(greenLedPin, LOW);
  digitalWrite(yellowLedPin, HIGH);
  digitalWrite(redLedPin, LOW);
  delay(1000);

  // Feu rouge pendant 3 secondes.
  digitalWrite(greenLedPin, LOW);
  digitalWrite(yellowLedPin, LOW);
  digitalWrite(redLedPin, HIGH);
  delay(3000);
}
```

4. Fonctionnement du programme

Les trois broches sont configurées comme sorties. Pour chaque phase, le programme place une seule sortie à HIGH et force les deux autres à LOW. `delay(3000)` maintient le vert ou le rouge pendant 3000 ms, tandis que `delay(1000)` maintient le jaune pendant 1000 ms. À la fin de `loop()`, Arduino recommence automatiquement au feu vert. Un cycle complet dure 7 secondes.

5. Résultats attendus

Intervalle du cycle	Vert	Jaune	Rouge
0 à 3 s	Allumé	Éteint	Éteint
3 à 4 s	Éteint	Allumé	Éteint
4 à 7 s	Éteint	Éteint	Allumé

Le cycle reprend ensuite sans arrêt et deux couleurs ne doivent jamais rester allumées simultanément.

6. Résultats expérimentaux et analyse

Capture(s) de simulation à insérer ici

Présenter les états vert, jaune et rouge, avec des légendes lisibles.

Observations réelles à compléter :

Durées observées : [...]

Ordre des couleurs : [...]

Comparaison avec le cycle attendu : [...]

7. Conclusion de l'exercice

Le programme construit un cycle séquentiel en commandant explicitement les trois sorties. Les temporisations prévues sont de 3 s, 1 s et 3 s, dans l'ordre vert-jaune-rouge. Cet exercice permet de comprendre la commande de plusieurs sorties et l'effet bloquant de `delay()`. La conformité visuelle et temporelle reste à confirmer dans Proteus. Difficulté éventuellement rencontrée : **[à compléter après la simulation]**.

Exercice 3 — Commande manuelle d'éclairage

Objectif : allumer une LED bleue uniquement pendant le maintien du bouton-poussoir.

1. Composants et connexions

Le montage emploie un Arduino Uno R3, un bouton BUTTON, une LED bleue, une résistance MINIRES de 10 kΩ pour le pull-down et une résistance MINIRES de 220 Ω pour limiter le courant de la LED.

Élément	Connexion Proteus
Bouton	Une borne vers 5 V ; l'autre vers la broche 2
Pull-down	Résistance 10 kΩ entre la broche 2 et GND
LED bleue	Broche 7 → résistance 220 Ω → anode ; cathode → GND

2. Schéma électrique

Capture du schéma Proteus à insérer ici

Montrer clairement le 5 V, le bouton, la broche 2, la résistance pull-down de 10 kΩ vers GND, ainsi que la LED sur la broche 7.

3. Programme Arduino commenté

```
const int buttonPin = 2;
const int ledPin = 7;

void setup() {
  // Le bouton utilise une résistance pull-down externe de 10 kΩ.
  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // La lecture vaut HIGH uniquement pendant l'appui.
  int buttonState = digitalRead(buttonPin);

  if (buttonState == HIGH) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }
}
```

4. Fonctionnement du programme

Lorsque le bouton est relâché, la résistance de 10 kΩ relie l'entrée 2 au GND. `digitalRead(2)` renvoie alors LOW et la LED est éteinte. Pendant l'appui, le bouton relie l'entrée au 5 V : la lecture devient HIGH et le programme allume la sortie 7. L'utilisation de INPUT est volontaire, car le pull-down demandé est une résistance externe. Aucun état n'est mémorisé après le relâchement.

5. Résultats attendus

État du bouton	Niveau sur D2	LED bleue
Relâché	LOW	Éteinte
Maintenu enfoncé	HIGH	Allumée

6. Résultats expérimentaux et analyse

Capture(s) de simulation à insérer ici

Montrer l'état relâché puis l'état maintenu, avec l'état correspondant de la LED bleue.

Observations réelles à compléter :

Comportement au repos : [...]

Comportement pendant l'appui : [...]

Comportement au relâchement : [...]

7. Conclusion de l'exercice

Cet exercice montre comment une entrée numérique commande directement une sortie. La résistance pull-down évite un niveau indéterminé lorsque le bouton est ouvert. Le résultat prévu est une LED allumée seulement pendant l'appui, sans basculement ni mémorisation. Ce comportement reste à vérifier dans Proteus. Difficulté éventuellement rencontrée : **[à compléter après la simulation]**.

Exercice 4 — Contrôle angulaire d'un servomoteur

Objectif : convertir la valeur 0–1023 d'un potentiomètre en un angle 0–180° avec `map()`, puis transmettre cette consigne au servomoteur raccordé à la broche 9.

1. Composants et connexions

Le montage comprend un Arduino Uno R3, un potentiomètre POT-HG et un servomoteur MOTOR-SERVO.

Élément	Connexion Proteus
Potentiomètre	Bornes extérieures vers 5 V et GND ; curseur vers A0
Signal du servomoteur	Broche PWM 9
Alimentation du servomoteur	VCC vers 5 V ; GND vers la masse commune

Pour une réalisation physique, un servomoteur peut nécessiter une alimentation 5 V externe capable de fournir le courant demandé. La masse de cette alimentation doit rester commune avec le GND de l'Arduino.

2. Schéma électrique

Capture du schéma Proteus à insérer ici

La capture doit montrer A0, la broche PWM 9, le potentiomètre, le servomoteur, le 5 V et le GND commun.

3. Programme Arduino commenté

```
#include <Servo.h>

const int potentiometerPin = A0;
const int servoPin = 9;

Servo myServo;

void setup() {
  // Association du signal du servomoteur à la broche 9.
  myServo.attach(servoPin);
}

void loop() {
  // Lecture de la position du potentiomètre.
  int analogValue = analogRead(potentiometerPin);

  // Conversion de 0-1023 vers un angle de 0-180 degrés.
  int angle = map(analogValue, 0, 1023, 0, 180);

  // Application de l'angle au servomoteur.
  myServo.write(angle);
}
```

4. Fonctionnement du programme

La bibliothèque Servo.h produit automatiquement le signal de commande adapté. attach(9) associe l'objet servomoteur à la broche 9. Ensuite, analogRead(A0) mesure la position du potentiomètre. La fonction map(analogValue, 0, 1023, 0, 180) transforme cette mesure en angle. Enfin, write(angle) applique la nouvelle consigne. La boucle recommence continuellement afin de suivre les déplacements du potentiomètre.

5. Résultats attendus

Valeur analogique	Angle théorique
0	0°
environ 512	environ 90°
1023	180°

Le déplacement du servomoteur doit suivre la variation du potentiomètre entre les deux positions extrêmes.

6. Résultats expérimentaux et analyse

Capture(s) de simulation à insérer ici

Montrer de préférence les positions proches de 0° , 90° et 180° .

Observations réelles à compléter :

Position minimale observée : [...]

Position médiane observée : [...]

Position maximale observée : [...]

Écart ou comportement particulier : [...]

7. Conclusion de l'exercice

Cet exercice associe une entrée analogique à une commande de position. L'utilisation de `map()` adapte correctement les deux plages : 0–1023 devient 0– 180° . Le programme doit donc permettre un suivi en temps réel du potentiomètre par le servomoteur. Les angles réellement affichés restent à relever dans Proteus. Difficulté éventuellement rencontrée : **[à compléter après la simulation]**.

Difficultés rencontrées

Cette partie doit décrire uniquement les difficultés réellement rencontrées pendant le câblage, la compilation ou la simulation, ainsi que la solution appliquée. Ne conserver aucun exemple qui ne correspond pas au travail effectué.

Difficulté 1 : [...]

Cause identifiée : [...]

Solution appliquée : [...]

Difficulté 2 : [...]

Cause identifiée : [...]

Solution appliquée : [...]

Conclusion générale

Les quatre programmes réalisés couvrent les principales fonctions demandées : lecture analogique, comparaison à un seuil, commande numérique temporisée, lecture d'un bouton avec pull-down et pilotage d'un servomoteur. Les broches et les valeurs temporelles choisies correspondent au sujet : A0 et D8 pour le premier exercice, D13-D12-D11 et 3 s-1 s-3 s pour le feu, D2 et D7 pour la commande manuelle, puis A0 et D9 pour le servomoteur.

La partie logicielle est prête à être compilée. La validation complète du TP nécessite maintenant la réalisation des quatre schémas dans Proteus, le chargement des fichiers HEX, l'exécution des essais et l'ajout des captures et observations réelles dans ce rapport.

Validation finale avant remise

- [] Nom, prénom, filière, année universitaire et date renseignés ;
- [] quatre captures de schémas lisibles insérées ;
- [] résultats réels et analyses complétés ;
- [] difficultés réelles décrites ;
- [] fichier exporté au format PDF sous le nom demandé.